

Week 6 - Lecture 11 - Problem: Graphical Projectile Simulation

ILO

- **KU3:** Design algorithms to solve simple programming problems.
- **CS1:** Structure small programs using iterations, functions, classes, and methods.
- **CS2:** Structure larger programs into manageable and reusable units.
- **CS3:** Write readable, descriptive, and well-documented program code.
- **CS5:** Express mathematical formulas as programming expressions and algorithms.
- **CS6:** Build basic interactive programs with text-based (and graphical) user interfaces.
- **CS9:** Use standard libraries and follow good programming practices.

Objectives

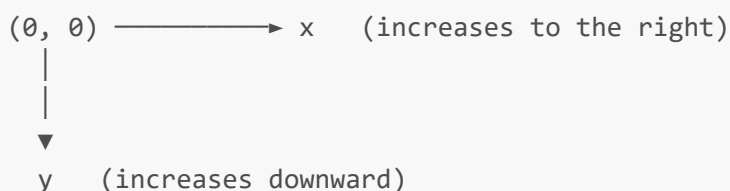
After completing this problem, you should be able to:

1. Create a window and draw points and circles with the `graphics` library.
2. Explain the default window coordinate system, where `(0, 0)` is the top-left corner and y increases downward.
3. Explain that `move(dx, dy)` is relative: it adds a change to the current position.
4. Use `setCoords` to create a coordinate system in which y increases upward.
5. Convert simulation values into the units used by a window.
6. Animate a projectile by moving a graphical object once per simulation step.

Background

In Lecture 10, a `Cannon` fired a projectile and displayed it in the terminal. In this problem, the projectile is displayed as a circle in a graphical window.

The `graphics` library draws in a window with its own coordinate system. By default:



This differs from the physics model, where y normally increases upward. Moving an object in the window therefore requires care.

You may reuse the constants and the update model from the earlier projectile problems. The goal is to practice coordinate systems and animation, not to build a precise physics model.

Task

Part 1: Create a window and draw shapes

Create a window and wait for a key press so that it does not close immediately:

```
from graphics import *

win = GraphWin("My Window", 600, 400) # width (x), height (y)
win.getKey()
```

Then draw a red circle and a green circle:

```
p1 = Point(40, 80)
p2 = Point(300, 10)

c1 = Circle(p1, 10)
c1.setFill("red")
c2 = Circle(p2, 5)
c2.setFill("green")

c1.draw(win)
c2.draw(win)
win.getMouse()
```

Observe where each circle appears. Which corner of the window is (0, 0)?

Part 2: Understand move

Move a circle in a loop and print its center with `getCenter()`:

```
for i in range(30):
    c1.move(1, 1)
    print(c1.getCenter())
    time.sleep(0.1)
```

Run it again with `c1.move(i, i)`. Compare the printed centers and explain why the movement is no longer steady.

Part 3: Set up a coordinate system

Create the window in the `Cannon` initializer and give it a coordinate system in which (0, 0) is the bottom-left corner and y increases upward:

```
self.win = GraphWin("Projectile Simulation", 600, 600)
self.win.setCoords(0, 0, 100, 100)
```

The four values are the lower-left x and y, followed by the upper-right x and y. After this call, positions and movements use these units rather than pixels, and no sign flip is needed for y.

Part 4: Fire the projectile graphically

Create a `Cannon` class that has a `Projectile` and a window. In `fire(velocity, theta)`:

1. Calculate the initial velocity components with `math.cos` and `math.sin`.
2. Create a circle at the starting position and draw it in the window.
3. While the projectile is at or above the ground, calculate the change in position for the current step.
4. Update the projectile's position and velocity.
5. Move the circle by the same change in position.
6. Wait briefly with `time.sleep`.

Use this update model:

```
dx = x_vel * dt
dy = y_vel * dt

self.projectile.x_pos += dx
self.projectile.y_pos += dy

x_vel -= AIR_DRAG * dt
y_vel -= G * dt

circle.move(dx, dy)
```

Use `G = 9.82`, `AIR_DRAG = 0.4`, and `dt = 0.02`.

Part 5: Choose a visible scale

With a small launch speed, the movement can be too small to see in a window that is 100 units wide. Experiment with the launch speed and, if you wish, a `SCALE` factor:

```
cannon.fire(20, math.radians(45))
```

Decide whether `SCALE` converts simulation units to window units, and use it consistently for both x and y so that the trajectory keeps its true shape.

Requirements

- Create the window with `GraphWin` and set its coordinates with `setCoords`.
- Draw the projectile as a `Circle`.
- Update the circle with `move(dx, dy)` using the change in position, not the absolute position.
- Use the same scale for x and y.
- Stop the loop when the projectile reaches the ground.
- Keep `G` and `AIR_DRAG` as module-level constants.

- Wait for a key press or mouse click at the end so the window does not close immediately.

Discussion Questions

1. In the default window coordinate system, where is `(0, 0)` and in which direction does y increase?
2. Why does `c1.move(i, i)` give a different result from `c1.move(1, 1)`?
3. What does `setCoords(0, 0, 100, 100)` change, and why is the y sign flip no longer needed?
4. Why should x and y use the same scale?
5. Why can a trajectory be too small to see, even when the code is correct?
6. What is the difference between the simulation time step `dt` and the pause passed to `time.sleep`?

Extension Challenges

1. Draw a small ground line and make the projectile stop when it reaches it. Account for the circle's radius.
2. Leave a trail by drawing a small `Point` or `Circle` at each step.
3. Use `getMouse()` to choose the launch angle from the position of a mouse click.
4. Fire a `Bomb` and a `Missile` from Lecture 10, using a different circle color for each.